

String parsing, metoder og funktioner

💡 Dagens fokus

- Fra `.includes()` til regular expressions: genkend mønstre i brugerens spørgsmål
- Hvornår logik hører hjemme i en funktion, og hvornår i en metode



8:15	5	🎓	Intro & agenda
8:20	10	🎓	<u>Repetition: .includes(), .startsWith(), .endsWith()</u>
8:30	20	💡	<u>Demo: Vis interessante løsninger</u>
8:50	10	👥	<u>Hvornår bliver I begrænsede af de funktioner?</u>
9:00	25	🎓	PROSA oplæg
9:25	10	🎓	<u>Regular expressions intro</u>
9:35	15	🔗	<u>Brug regex til at søge i VS Code</u>
9:50	20	🕒	Pause
10:10	10	🎓	<u>Brug regex på strings med .search()</u>
10:20	25	🔧	<u>Udvid jeres søgelogik med regex</u>
10:45	15	💡	<u>Demo: Vis interessante løsninger</u>
11:00	10	🎓	<u>Metoder vs. funktioner — hvornår bruger vi hvad?</u>
11:10	10	🔗	<u>Identificér funktioner vs metoder i jeres AMAbots</u>
11:20	10	🔧	Opgave: funktion der tilføjer prefix til svar

1. Repetition: `.includes()`, `.startsWith()`, `.endsWith()`

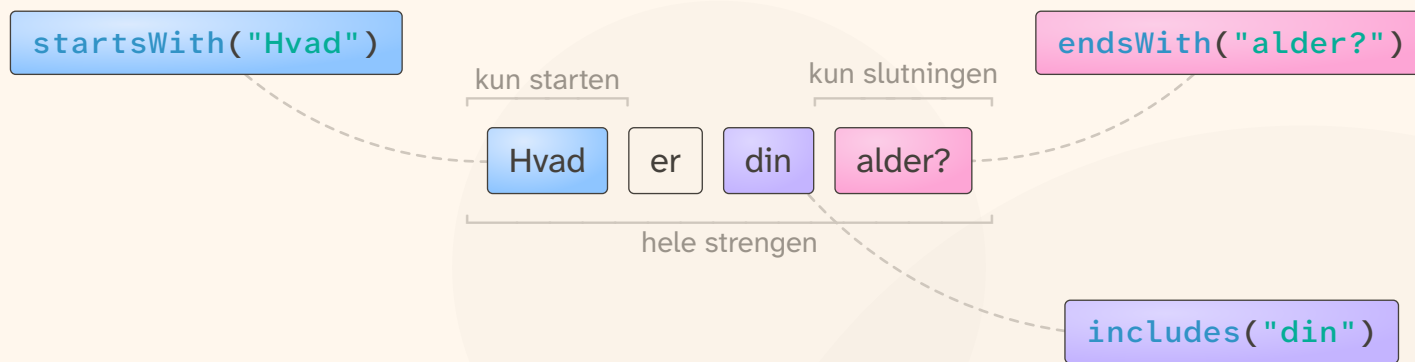
```
const input = "Hvad er din alder?"
```

<code>input.startsWith("Hvad");</code>	<code>true</code>	matcher starten af strengen
<code>input.startsWith("hvad");</code>	<code>false</code>	der er forskel på store og små bogstaver
<code>input.toLowerCase().startsWith("hvad");</code>	<code>true</code>	input normaliseres først
<code>input.endsWith("alder?");</code>	<code>true</code>	matcher slutningen af strengen
<code>input.endsWith("alder");</code>	<code>false</code>	skal matche præcist på slutningen
<code>input.includes("din");</code>	<code>true</code>	hvor som helst i strengen

Alle tre returnerer en **boolean** – **true** eller **false** – så de kan bruges direkte i en **if**-sætning.

```
if (input.includes("alder")) {  
    return "Jeg er 25 år";  
}
```

1.1 Hvor i strengen kigger hver metode?



⚠ Bemærk:

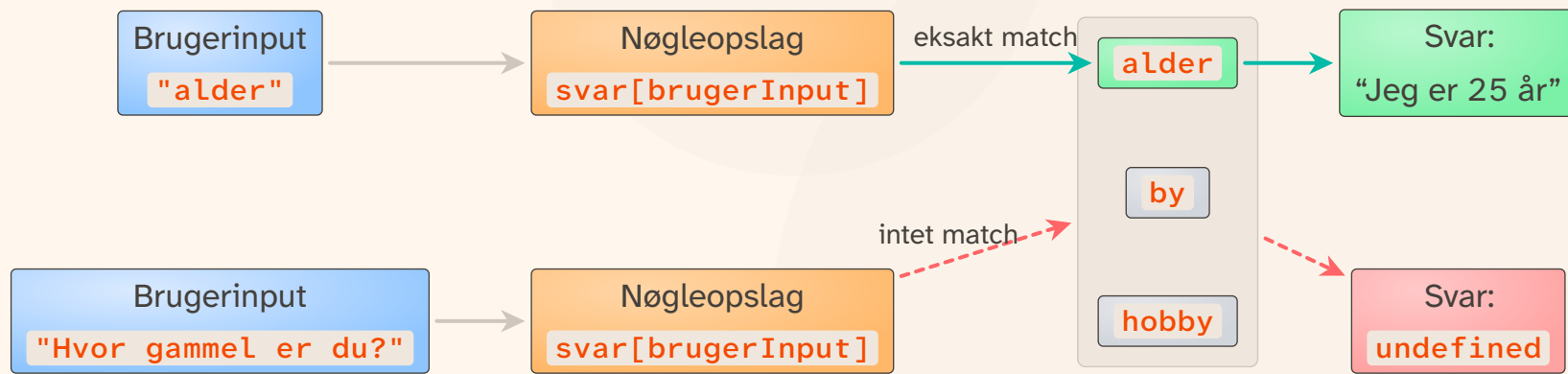
- De er **case sensitive**: `"Alder".includes("alder")` er `false`. Normalisér altid først med `.trim()` og `.toLowerCase()`
- De tager **tekst**, ikke **regex**: `"abc".startsWith(/a/)` kaster en `TypeError`
- De siger kun **om** der er et match — ikke **hvor** det er, og ikke **hvad** der står

1.2 De naive opslag

Oprindeligt slog I svaret op direkte med `svar[input]` – brugeren skulle skrive præcis det forventede input.

Med stringmetoderne kan I i stedet lede efter **dele** af det, brugeren skrev.

```
const svar = {  
  alder: "Jeg er 25 år",  
  by: "Jeg bor i Aarhus",  
  hobby: "Jeg kan godt lide at spille fodbold",  
}
```



2. Demo: Vis interessante løsninger

🔗 Vi kigger på nogle af jeres AMAbots

Hvert bord udvælger et eksempel fra en af jeres løsninger, og viser den frem på skærmen.

🔍 Læg blandt andet mærke til:

- 01 **Normalisering:** Hvordan ryddes der op i input? `trim()`, `toLowerCase()` — eller noget helt tredje?
- 02 **Datastruktur:** Hvordan er synonymerne gemt? Et array per emne, ét stort objekt, flere objekter?
- 03 **Rækkefølge:** Hvilket tjek kommer først — og hvad sker der, hvis man bytter om?
- 04 **Fallback:** Hvad svarer botten, når den ikke forstår spørgsmålet?

3. Hvornår bliver I begrænsede af de funktioner?

Grupper ⌚ 10 min

🗨 Diskutér i grupper:

Find **mindst tre** situationer, hvor `includes()`, `startsWith()` og `endsWith()` ikke rækker til jeres AMAbot. Tag udgangspunkt i jeres egen kode — og de løsninger, I lige har set.

🗨 Diskutér for eksempel:

- **Store og små bogstaver:** Hvor mange steder er I nødt til at kalde `toLowerCase()`?
- **Tegn, ikke ord:** `"hobby".includes("by")` og `"katastrofe".includes("kat")` er begge `true`
- **Synonymer:** Hvor lang bliver `if`-sætningen, når “alder”, “gammel”, “født” og “år” alle skal virke?
- **Hvor er matchet?** Metoderne siger kun `true` eller `false`, ikke hvor i strengen ordet står — er det et problem?
- **Hvad står der?** I kan ikke hive en værdi ud — fx årstallet i “er du født i 1999?”

📋 Fælles opsamling bagefter — hvad fandt I?

4. Regular expressions intro

En **regular expression** (regex, eller på dansk et “regulært udtryk”, if you must) er et mønster, ikke en tekst. Det beskriver, hvordan noget ser ud, ikke (nødvendigvis) præcis hvad der står.

I JavaScript skrives mønsteret mellem to skråstreger:

<code>/alder/</code>	ren tekst — matcher det samme som <code>.includes(“alder”)</code>
<code>/hund kat fisk/</code>	alternation: hund ELLER kat ELLER fisk
<code>/[0-9]+/</code>	tegnklasse + “en eller flere”: et helt tal
<code>/[a-zæøå]+/</code>	tegnklasse: et eller flere små bogstaver
<code>/\balder\b/</code>	<code>\b</code> = ordgrænse: “alder”, men ikke “aldersgrænse”
<code>/^hvad/</code>	<code>^</code> = start på strengen — ligesom <code>.startsWith(“hvad”)</code>
<code>/alder\$/</code>	<code>\$</code> = slut på strengen — ligesom <code>.endsWith(“alder”)</code>
<code>/kæledyr/i</code>	<code>i</code> -flag: case-insensitive, matcher både “kæledyr”, “Kæledyr”, “KÆLEDYR” osv.
<code>/hvad hvor/g</code>	<code>g</code> -flag: find alle match, ikke kun det første

4.1 Regex-pendanterne til `includes()`, `startsWith()` og `endsWith()`

`test()` er regexets svar på `includes()`: den tager en streng og returnerer `true` eller `false`.

```
const input = "hvad er din alder?";
```

<code>input.startsWith("hvad");</code>	<code>/^hvad/.test(input)</code>
<code>input.endsWith("?");</code>	<code>/\?\$/.test(input)</code>
<code>input.includes("din");</code>	<code>/din/.test(input)</code>

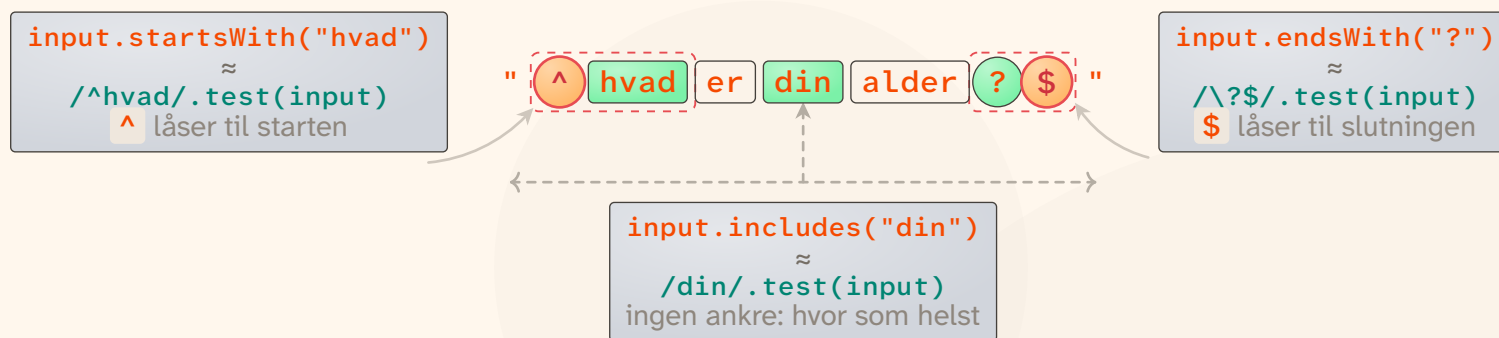
💡 Hvorfor så overhovedet skifte?

- **i**-flaget erstatter `toLowerCase()`: `/^hvad/i.test(input)` virker på “Hvad”, “HVAD” og “hvad” — uden at I rører ved input
- **Alternation** erstatter `||`-kæder: ét mønster `/\b(alder|gammel|født|år)\b/i` i stedet for fire `includes()`-kald
- **Ordgrænser** fjerner de falske match: `/\bby\b/` rammer “min by”, men ikke “hobby”

💡 Afprøv regex i browseren

- Byg og test mønstrene på regex101.com — vælg “Flavor: JavaScript”, så I tester den korrekte “dialekt”

4.2 Ankrene `^` og `$` holder mønsteret fast i enderne



💡 Uden anker flyder mønsteret frit

- `/hvad/` matcher "hvad" **hvor som helst** i strengen — det er `includes()`
- `/^hvad/` kan kun matche, hvis strengen **begynder** sådan, og `/\?$ /` kun hvis den **slutter** sådan
- `\?` med backslash foran: `?` er et specialtegn i regex, så det skal "escapes", når vi mener et rigtigt spørgsmålstegn

4.3 Regex-cheatsheet: Et fornuftigt udgangspunkt

Mønster	Betyder
<code>abc</code>	Teksten “abc”, tegn for tegn
<code>.</code>	Ét vilkårligt tegn
<code>[aeiou]</code>	Ét af tegnene i klassen
<code>[^0-9]</code>	Ét tegn, der <i>ikke</i> er i klassen
<code>[a-zæøå]</code>	Interval — husk selv at tage æøå med
<code>\d</code>	Ét ciffer — samme som <code>[0-9]</code>
<code>a b</code>	<code>a</code> eller <code>b</code> (alternation)
<code>\b</code>	Ordgrænse: <code>/\bby\b/</code> rammer ikke “hobby”

Mønster	Betyder
<code>^</code>	Start på strengen — som <code>startsWith()</code>
<code>\$</code>	Slut på strengen — som <code>endsWith()</code>
<code>+</code>	Én eller flere af det foregående
<code>*</code>	Nul eller flere
<code>?</code>	Nul eller én — gør det foregående valgfrit
<code>\d{2}</code>	Præcis to cifre
<code>(...)</code>	Gruppe — genbruges som <code>\$1</code> , <code>\$2</code>
<code>/.../i</code>	Flag: ignorér store og små bogstaver
<code>/.../g</code>	Flag: find alle match, ikke kun det første

Læs det som en sætning: `/^[A-ZÆØÅ][a-zæøå]+sen$/` = “start, ét stort bogstav, et eller flere små bogstaver, `sen`, slut”.

5. Brug regex til at søge i VS Code

♥ Parvis ⌚ 15 min

🔍 Sådan søger I med regex i VS Code

Vi laver i fællesskab et nyt tekstdokument med alle jeres navne, ved at kopiere fra Canvas og rense med regexes.

Alle opgaverne løses ved at **søge** — I skal ikke skrive kode.

- `Cmd + F` søger i den åbne fil · `Cmd + Shift + F` søger i alle filer i mappen
- Slå regex til med `. *`-knappen i søgefeltet
- Fold erstat-feltet ud med pilen til venstre. Her kan I genbruge capture-grupper som `$1` og `$2`

♥ Find sammen med jeres makker:

- 01 Alle navne, der begynder med `S`
- 02 Alle navne, der ender på `sen`
- 03 Alle linjer med præcis to navne
- 04 Alle linjer med præcis fire navne
- 05 Alle navne, der indeholder `æ`, `ø` eller `å`
- 06 Byt om på fornavn og efternavn: søg `^(\\S+) (\\S+)$`, erstat med `$2, $1`
- 07 ...og afprøv tilpasninger, så det fungerer godt for flere efternavne

6. Brug regex på strings med `.search()`

```
const text = "Hvor gammel er du egentlig?";  
text.search(/gammel/i); // 5 – indeks på første match  
text.search(/kæledyr/i); // -1 – intet match
```

```
// Ordgrænser fjerner de falske match fra includes()  
"hobby".search(/\bby\b/); // -1
```

```
// Ét regex mønster per emne  
const patterns = {  
  alder: /\b(alder|gammel|født)\b/i,  
  by: /\b(bor|by|adresse)\b/i,  
  kæledyr: /\b(hund|kat|kæledyr)\b/i,  
};
```

⚠ `search()` returnerer et tal – ikke en boolean

Et **indeks** (0 eller højere), hvis mønsteret findes, og `-1` hvis det ikke gør. Derfor skriver vi `!== -1` og ikke

`if (text.search(...))` – indeks `0` er nemlig falsk, så et match helt i starten af strengen ville blive læst som “intet match”.

6.1 match() – hvad står der egentlig i strengen?

```
const input = "hvad er din alder og din by";  
const et = input.match(/din ([a-zæøå]+)/);  
et[0];      // "din alder"    – hele matchet  
et[1];      // "alder"        – capture-gruppen i ( )  
et.index;   // 8              – hvor matchet starter
```

```
const alle = input.match(/din ([a-zæøå]+)/g);  
alle;      // ["din alder", "din by"] – kun strenge, ingen grupper
```

```
input.match(/kæledyr/); // null – husk at tjekke!
```

⚠ **match() returnerer tre helt forskellige ting**

- **Et array med detaljer** (uden **g**), **et array af strenge** (med **g**) eller **null**
- Derfor: gem resultatet i en variabel, tjek for **null**, og brug det først derefter

6.2 Sæt det sammen: `search()` finder emnet, `match()` henter detaljen

```
function analyzeQuestion(input) {  
  const text = input.trim().toLowerCase();  
  let subject = null;  
  // search(): hvilket emne handler det om?  
  for (const key in patterns) {  
    if (text.search(patterns[key]) !== -1) {  
      subject = key;  
      break;  
    }  
  }  
  // match(): er der en konkret detalje at hive ud?  
  let year = text.match(/\\b(19|20)\\d{2}\\b/);  
  if (year !== null) {  
    year = year[0];  
  }  
  return { subject, year };  
}
```

7. Udvid jeres søgelogik med regex

⚙ Solo ⌚ 25 min

📖 Trin for trin:

- 01 Lav en branch: `git switch -c feature/regex-matching`
- 02 Byg et `mønstre`-objekt, hvor hvert emne har ét regulært udtryk med alternation:
`alder: /\b(alder|gammel|født)\b/i`
- 03 Erstat jeres `includes()`-tjek med `search(...) !== -1`
- 04 Tjek, at ordgrænserne (`\b`) fjerner de falske match — prøv fx `"hobby"` mod by-mønsteret
- 05 Brug `^` eller `$` mindst ét sted — fx til at genkende, om spørgsmålet slutter med `?`
- 06 Brug `match()` til at hive én konkret detalje ud af et spørgsmål — fx et årstal, et tal eller et navn
- 07 Husk `null`-tjekket, før I bruger resultatet af `match()`
- 08 Commit undervejs

🔧 Test det:

Skriv **fem forskellige formuleringer af samme spørgsmål** ind i jeres form input, og notér om AMAbotten finder det rigtige emne.

8. Demo: Vis interessante regex-løsninger

🔗 Anden runde — nu med mønstre

Vi kigger på et par af de `mønstre`-objekter, I lige har bygget, og afprøver dem med nogle drilske spørgsmål i fællesskab.

Hvert bord udvælger et eksempel fra en af jeres løsninger, og viser det frem på skærmen.

🔍 Hold øje med:

- 01 **Læsbarhed:** Kan I gætte, hvad mønsteret gør, uden at få det forklaret?
- 02 **Hvor bredt eller smalt** er mønsteret? Hvad fanger det, som det ikke burde fange?
- 03 **Ordgrænser og ankre:** Hvor gjorde `\b`, `^` eller `$` en reel forskel?
- 04 `match()`: Hvilke detaljer lykkedes det jer at hive ud af et spørgsmål?

9. Metoder vs. funktioner – hvornår bruger vi hvad?

En **metode** er bare en funktion, der er gemt som property på et objekt — og som via `this` har adgang til objektets egne data. Teknisk er der ingen forskel; forskellen er, hvor logikken bor.

```
// Selvstændig funktion: får alt, den skal bruge, ind som argumenter
```

```
function normalize(input) {  
    return input.trim().toLowerCase();  
}
```

```
// Metode: en funktion gemt som property på et objekt
```

```
const amabot = {  
    profile: { navn: "Peter", alder: 25, by: "Aarhus" },  
    patterns: { alder: /\b(alder|gammel)\b/i, by: /\b(bor|by)\b/i },  
    answer(input) {  
        const subject = findSubject(normalize(input), this.patterns);  
        if (subject === null) return "Det kan jeg ikke svare på endnu";  
        return `Min ${subject} er ${this.profile[subject]}`;  
    },  
};  
amabot.answer("Hvor gammel er du?"); // => "Min alder er 25"
```

9.1 Tommelfingerregler: Hvor lægger vi logikken?

👍 Vælg en selvstændig funktion, når:

- Logikken kun afhænger af sine argumenter
- Den skal genbruges flere steder
- Den er let at teste isoleret
- Fx `normalizeInput()`, `chooseRandom()`

⚠️ `this` er den sædvanlige fælde

- Inde i en metode peger `this` på objektet — `this.profile`, `this.patterns`
- Skriv metoder med den korte syntaks (`answer(input) { ... }`) eller `function` — en **arrow function** som metode har ikke sit eget `this`

👍 Vælg en metode, når:

- Logikken arbejder på objektets egen data
- Den hører begrebsmæssigt til objektet
- Kaldet læses bedre som `amabot.answer(...)`
- Fx `amabot.answer()`, `amabot.remember()`

10. Identificér funktioner vs metoder i jeres AMAbots

♥ Parvis ⌚ 15 min

🗨 Forklar for hinanden:

- 01 Åbn jeres **egen** AMAbot-kode, og find alle de steder, hvor noget bliver kaldt: `noget(...)` eller `noget.andet(...)`
- 02 Skriv en kommentar ved hvert kald, om det er en **funktion** eller en **metode** — og om I selv har skrevet den, eller den er indbygget i JavaScript
- 03 Byt: forklar din makker, hvorfor du har klassificeret dem sådan. Det er forklaringen, der er øvelsen

💡 Tommelfingerreglen:

Hører logikken til dataen? → metode. Er den generel og uafhængig? → funktion.

11. Opgave: funktion der tilføjer prefix til svar

⚡ Solo ⌚ 20 min

```
function chooseRandom(list, fallback = "Godt spørgsmål!") {  
  if (list.length === 0) return fallback;  
  return list[Math.floor(Math.random() * list.length)];  
}  
  
const openings = ["Godt spørgsmål!", "Ah ja, jeg kan godt forstå at du spørger."];  
  
// Wrapper: tager et færdigt svar ind, og giver et pyntet svar tilbage  
const withOpening = (answer) => `${chooseRandom(openings)} ${answer}`;  
return withOpening(answerQuestion(input)); // "Godt spørgsmål! Jeg er 25 år"
```

Skriv en variant af `withOpening()` i jeres egen kode, kald den lige før svaret sendes til brugeren — og udvid `openings` med mindst fem vendinger. Færdiggør det hjemme.

💡 Hvorfor en wrapper?

- **Indpakning:** `withOpening()` regner ikke selv noget ud — den pynter kun på det, en anden funktion returnerede
- Skal botten lyde formel i stedet for hyggelig, ændrer I kun listen, ikke svarlogikken

